

WORKSHEET 1: INTRO INTO REPRODUCIBILITY

Kosmas Hench
2025-05-26

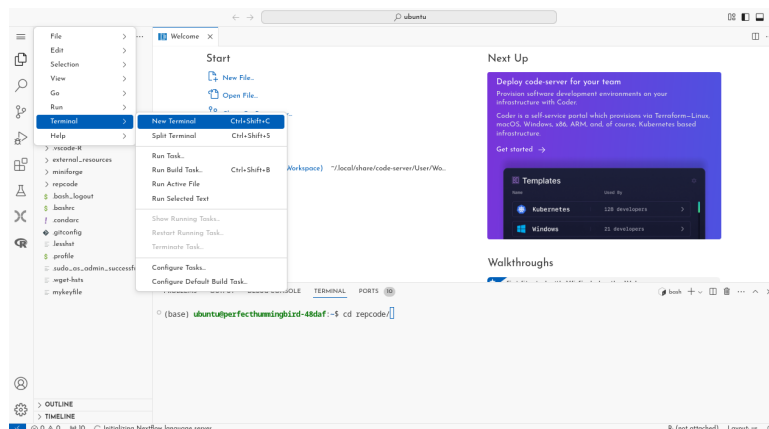
CONTENTS

- 1 Setup 1
- 2 PCA and Clustering 2
- 3 Plotting 5

As a practical example of reproducible scripting, we are going to look at a bunch of penguins. [Gorman et al. 2014](#) provide a data set with of morphological traits of three species of penguins from the Palmer Archipelago at the Antarctic Peninsula (Adélie penguins, *Pygoscelis adeliae*; chinstrap penguins, *P. antarctica* & gentoo penguins, *P. papua*). The aim of the example analysis is a descriptive comparison the samples, where we are going to explore if the three species are characterized by different morphologies. For this end we are going to apply a principal component analysis (PCA) to summarize the main variation in the data set. Additionally, we are going to run a *k*-means cluster analysis to see if the samples are grouped according to their species when clustered solely based on the measured morphological traits.

1. SETUP

The code base for this session is provided in the folder `repcode` within your `de.NBI $HOME` directory. To get there, please open the `VS code` terminal and enter the following code in the shell:



```
cd repcode
```

The available data contains information on 344 samples. Records include the species, sampling site, four quantitative morphological measurements, the samples sex and year of sampling. The data is provided in a `tsv` file (`data/penguins.tsv`) with the following structure:

sample id	species	island	bill length (mm)	bill depth (mm)	flipper length (mm)	body mass (g)	sex	year
sample_01	adelie	torgersen	39.1	18.7	181	3750	male	2007
sample_02	adelie	torgersen	39.5	17.4	186	3800	female	2007
...	...	...	...	...	...	...	...	...
sample_277	chinstrap	dream	46.5	17.9	192	3500	female	2007
sample_278	chinstrap	dream	50.0	19.5	196	3900	male	2007
...	...	...	...	...	...	...	...	...
sample_153	gentoo	biscoe	46.1	13.2	211	4500	female	2007
sample_154	gentoo	biscoe	50.0	16.3	230	5700	male	2007
...	...	...	...	...	...	...	...	...

The analysis is distributed over two scripts: First we will use `code/py/clustering.py` to run the PCA and the k-means clustering. The script will create the new directory `results` and export its results to tables therein. Then we will run `code/R/plotting.R` to plot the results. The final product of both scripts will be the figure `results/cluster_results.pdf`.

Both scripts can be executed from the command line and should run fairly quickly (< 1 minute). Below, the scripts are broken down into their main steps.

2. PCA AND CLUSTERING

We start with the cluster analysis within the script `clustering.py`. It can be run using:

```
python code/py/clustering.py
```

The script starts with a header section in which all required `python` packages are loaded. Then we fix the seed to enable the reproducibility of the random aspects of this script.

```
1 import session_info          # to capture the computing environment
2 import os                    # to create target directories
3 import pandas as pd          # to help organize data
4 import numpy as np           # for number crunching
5 from sklearn.decomposition import PCA      # PCA method
6 from sklearn.cluster import KMeans        # k-means clustering method
7 from sklearn.preprocessing import StandardScaler # to normalize data
8
9 # Set the random seed for reproducibility
10 np.random.seed(42)
```

Next, the external data (`penguins.tsv`) is imported.

To adhere to the requirements of our planned analysis we need to do some data cleaning though: Since PCAs can't operate on missing data, samples that have `NA` entries are dropped from the data set. As a form of housekeeping, we reset the *index* of our dataset to close gaps within the indices where the `NA` entries used to be (this will make our live easier later).

```
12 # Load data from TSV file, first three lines are comments
13 data = pd.read_csv('data/penguins.tsv', delimiter='\t', skiprows=3)
14
15 # The PCA can not operate on NA values
16 data = data.dropna()
17
18 # Reset the index of the original data DataFrame
19 data.reset_index(drop=True, inplace=True)
```

Then, we select the columns that we want to consider within the PCA. We first create an array with the column names of the relevant morphological measurements which we then use to subset the data.

After this, we standardize the morphological data using a *z-transformation*. This will shift the data so that its mean (μ) is 0, and its standard deviation (σ) is 1:

$$z = \frac{x - \mu}{\sigma}$$

(Another common transformation would be to re-scale the data to the range of 0–1. This could be done with the `python` function `MinMaxScaler()` instead of `StandardScaler()`.)

```
21 # store names of columns with morphology data
22 morph_col_names = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
23
24 # Select columns with morphology data for PCA
25 X = data[morph_col_names]
26
27 # Standardize the features
28 scaler = StandardScaler()
29 X_scaled = scaler.fit_transform(X)
```

Now, we can go ahead and actually run the PCA on the filtered and normalized data.

For this, we select the number of principal components that we want to report (`n_components=2`) and we fix the seed for the PCA run (by setting `random_state` to a fixed value that is governed by the overall seed set in line 10).

```
31 # Perform PCA (with reproducible seed)
32 pca = PCA(n_components=2, random_state=np.random.randint(1e6)) # We'll keep only the
    first 2 principal components for visualization
33 X_pca = pca.fit_transform(X_scaled)
```

Similar to the PCA, the k-means clustering is also conducted on the filtered and normalized data.

Here, we first set the number of clusters that we want to assign to three (for the three penguin species, `n_clusters=3`), and again we set the internal seed of the clustering method to a fixed value.

```
35 # Perform KMeans clustering, trying to detect the three species (with reproducible seed)
36 kmeans = KMeans(n_clusters=3, random_state=np.random.randint(1e6)) # You can adjust the
    number of clusters as needed
37 kmeans.fit(X_scaled)
38 cluster_labels = kmeans.labels_
```

At this point the main work of the two clustering analysis has been done.

The rest of the python script is concerned with the task of getting the results into a format that can be easily exported and plotted.

To this end, we first add the PCA scores and the label of the assigned cluster as new columns to the original (filtered) data to export the individual results.

```
40 # Combine clustering labels, PC1, PC2 values, and the original data into a DataFrame
41 combined_data = pd.concat([data,
42                             pd.DataFrame({'PC1': X_pca[:, 0],
43                                           'PC2': X_pca[:, 1],
44                                           'cluster': cluster_labels})], axis=1)
```

Then, we tackle the group-level results for the k-means clustering:

We start this by computing the centroid location of each identified cluster. As the coordinates for this are reported in terms of the normalized measures, we need to back-transform them to get the centroid coordinates relative to the real measured values. We also report the *inertia* as a measure of confidence for the clustering.

```
46 # Get cluster centers in original feature space
47 cluster_centers = kmeans.cluster_centers_
48
49 # Reverse the scaling operation to obtain centroids in the original feature space
50 cluster_centers_original = scaler.inverse_transform(kmeans.cluster_centers_)
51
52 # Get inertia (intra-cluster sum of squares)
53 inertia = kmeans.inertia_
```

Then, we convert the location of the cluster centroids from the measurement space into the PCA space. We are going to use this later to compare the assigned clusters with the actual species membership within the PCA space.

```
55 # Transform cluster centers to PCA space
56 cluster_centers_pca = pca.transform(cluster_centers_)
```

For the same reason, we compute several elements that are needed to generate an ellipse for each cluster in PCA space. We also need to take into account that angles are represented differently in python and in R (which we are going to use for plotting).

```
58 # Calculate semi-major and semi-minor axes lengths and rotation angle for each cluster
```

```

59 # to be able to draw ellipses for the clusters
60 semi_major_axes = []
61 semi_minor_axes = []
62 rotation_angles = []
63
64 for i, cluster_center in enumerate(cluster_centers_pca):
65     # Calculate covariance matrix for the cluster
66     cov_matrix = np.cov(X_pca[cluster_labels == i].T)
67     # Calculate semi-major and semi-minor axes lengths
68     eig_vals, eig_vecs = np.linalg.eig(cov_matrix)
69     semi_major_axes.append(np.sqrt(inertia / len(X_pca)) * np.sqrt(eig_vals[0]))
70     semi_minor_axes.append(np.sqrt(inertia / len(X_pca)) * np.sqrt(eig_vals[1]))
71     # Calculate angle of rotation
72     rotation_angle = np.degrees(np.arctan2(eig_vecs[1, 0], eig_vecs[0, 0]))
73     rotation_angles.append(rotation_angle)
74
75 # Adjust the angle of rotation to match R's coordinate system
76 rotation_angles_adjusted = [(angle + 180) % 360 for angle in rotation_angles]

```

Now, we have all the pieces that we want to report for the group-level results of the clustering analysis. To be able to export them as a clean tsv file, we bundle them up within a pandas data frame.

```

78 # Create a DataFrame to store the centroid positions and ellipse parameters
79 cluster_data = pd.DataFrame({
80     'cluster': range(len(cluster_centers)),
81     'centroid_{nm}'.format(nm = morph_col_names[0]) : cluster_centers_original[:, 0],
82     'centroid_{nm}'.format(nm = morph_col_names[1]) : cluster_centers_original[:, 1],
83     'centroid_{nm}'.format(nm = morph_col_names[2]) : cluster_centers_original[:, 2],
84     'centroid_{nm}'.format(nm = morph_col_names[3]) : cluster_centers_original[:, 3],
85     'centroid_PC1': cluster_centers_pca[:, 0],
86     'centroid_PC2': cluster_centers_pca[:, 1],
87     'semi_major_axis': semi_major_axes,
88     'semi_minor_axis': semi_minor_axes,
89     'rotation_angle': rotation_angles_adjusted
90 })

```

Finally, we can tackle the actual data export for all the results.

We first check if a results folder exists (if this is not the case, we create it).

Then, the file names for the individual- and group-based results are defined, and both data frames are exported as a tsv file.

```

92 # Create the "results" folder if it doesn't exist
93 if not os.path.exists('results'):
94     os.makedirs('results')
95
96 # Define the file paths for the TSV files
97 output_data = 'results/clustered_penguins.tsv'
98 output_cluster = 'results/clustered_centroids.tsv'
99
100 # Export the combined_data DataFrame to a TSV file
101 combined_data.to_csv(output_data, sep='\t', index=False)
102
103 # Export the cluster_data DataFrame to a TSV file
104 cluster_data.to_csv(output_cluster, sep='\t', index=False)

```

Here, the work within this script is done.

However, for the sake of reproducibility, we report the computing environment using `session_info.show()`. This will print out the used python version, as well as the versions of all required python packages.

```

106 # Log session information
107 print("python packages used:\n-----")
108 session_info.show()
109 print("-----\n")

```

3. PLOTTING

After having run the clustering analyses in `python`, we now turn to `R` to create a nice visualization of the results.

The `R` script can also be executed directly from the command line

```
Rscript code/R/plotting.R
```

Similar to the `python` script, the `R` script also starts with a header section to load all required `R` packages for this task and to set the seed for all random elements.

```
1 ## --- Setup section ---
2 # Load required libraries
3 suppressPackageStartupMessages(library(tidyverse)) # For data manipulation and plotting
4 library(prismatic) # For reading SVG images
5 library(patchwork) # For arranging plots
6 library(ggforce) # For plotting ellipses and circles
7 library(png) # For reading images
8
9 # Set seed for reproducibility
10 set.seed(42)
```

Within the header we also decide on the general appearance of the figure by setting the plotting theme and the used font.

```
9 fnt_sel <- "Arial"
10
11 # Set theme for plots
12 theme_set(theme_minimal(base_family = fnt_sel) %+replace%
13             theme(panel.border = element_rect(linewidth = .3,
14                                                 fill = "transparent"),
15                 legend.position = "none",
16                 plot.subtitle = element_text(hjust = .5,
17                                             margin = margin(b = 5))) )
```

The next bit of preparation is the definition of helper functions. We define those here already to declutter the part of the script where the plotting action actually happens.

The first custom function (`read_png()`) will allow us to import penguin artwork for the plot illustrations.

```
21 # Function to read png files
22 read_png <- function(file_path){
23   readPNG(file_path) >
24   grid::rasterGrob(interpolate=TRUE)
25 }
```

The second helper function (`reformat_names()`) will do some string operations to format the plot labels nicely.

```
27 # Function to reformat column names for better visualization
28 reformat_names <- \(x){
29   str_replace_all(x, "_", " ") >
30   str_replace(" ([a-z])*$", " \\(\\1\\)")
31 }
```

As a last piece of preparation, we create variables to hold the species names, the matching cluster IDs. We also set the color scheme to be used for the species and clusters.

```
33 # Define species names and corresponding cluster indices
34 species <- c("adelie", "chinstrap", "gentoo")
35 cluster_idx <- c(0, 2, 1) > set_names(nm = species)
36
37 # Define cluster names and colors
38 cluster_names <- species > set_names(nm = cluster_idx)
```

```

39 clrs <- c("#BC4743", "#76a8cc", "#0e6261") > # "#5b898b") > #c("#F26C0DFF", "#BE60C5FF", "#0A6B70FF") >
40   set_names(nm = species)
41 clrs_cluster <- set_names(clrs, nm = cluster_idx[names(clrs)])

```

With all this preparation in place, we can now turn towards the actual work.

The serious part starts with the import of the results from the `python` script (remember those `tsv` files?). Upon the import of the individual data (`clustered_penguins.tsv`), we add a small comparison where we check if the assigned cluster and the actual species actually match.

```

43 ## --- Loading Data ---
44 # Read data for clustered penguins and centroids
45 data_samples <- read_tsv("results/clustered_penguins.tsv",
46   col_types = "cccdcccdddi") >
47   mutate(correctly_assigned = cluster == cluster_idx[species])
48
49 data_centroids <- read_tsv("results/clustered_centroids.tsv",
50   col_types = "idccccccdd")

```

For the first panel of the final plot, we create a copy of the two data sets in *long format*. This will make it easier to facet over the different feature types.

```

52 # Convert data to long format for plotting along facets
53 data_samples_long <- data_samples >
54   pivot_longer(bill_length_mm:body_mass_g)
55
56 data_centroids_long <- data_centroids >
57   select(cluster:centroid_body_mass_g) >
58   set_names(nm = \{x\}{str_remove(x, "centroid_")}) >
59   pivot_longer(bill_length_mm:body_mass_g)

```

Now we can start with the actual plotting.

We are going to build a complex figure consisting of four sub-panels and an illustrated legend. To make this better digestible, we will construct each panel on its own and then assemble the final figure at the end.

So, we start with the first panel where we compare the individual measurements with the centroid location of the respective cluster for each morphology feature.

```

61 ## --- Main Plots ---
62 # Plot 1: Feature measurements
63 p1 <- data_samples_long >
64   ggplot(aes(y = species, x = value, color = species)) +
65   geom_jitter(height = 0.25, width = 0,
66     size = .5,
67     alpha = .3) +
68   geom_point(data = data_centroids_long,
69     aes(y = cluster_names[as.character(cluster)],
70       color = cluster_names[as.character(cluster)]),
71     size = 3, shape = 21, fill = clr_alpha("white", .75)) +
72   scale_color_manual(values = clrs) +
73   labs(subtitle = "feature measurements") +
74   facet_grid(~reformat_names(name),
75     scales = "free_x", switch = "x") +
76   theme(axis.title = element_blank(),
77     axis.text.y = element_blank(),
78     strip.placement = "outside")

```

The next panel is going to show the plain results of the PCA with the individuals colored by their actual species. We are going to add an ellipse for each species as well as highlights for those individuals that were wrongly assigned (for a comparison with the next panel...).

```

80 # Plot 2: PCA with actual species
81 p2 <- data_samples >
82   ggplot(aes(x = PC1, y = PC2, color = species)) +
83   geom_point(data = data_samples > filter(!correctly_assigned),
84             shape = 1, size = 3) +
85   geom_point(size = .5, alpha = .3) +
86   stat_ellipse(data = data_samples,
87               mapping = aes(x = PC1, y = PC2, color = species),
88               linewidth = .2) +
89   scale_color_manual(values = clr) +
90   labs(subtitle = "actual species") +
91   coord_fixed(xlim = c(-3,4),
92              ylim = c(-2.5,3),
93              ratio = 1)

```

Then, we create a panel with the same PCA results – but this time we color the individuals according to their assigned cluster based on the k-means clustering. Again, we add highlights for miss-assigned individuals to make a direct comparison easier.

```

95 # Plot 3: PCA with assigned clusters
96 p3 <- data_samples >
97   ggplot(aes(x = PC1, y = PC2, color = factor(cluster))) +
98   geom_point(data = data_samples > filter(!correctly_assigned),
99             shape = 1, size = 3) +
100  geom_point(size = .5, alpha = .3) +
101  scale_color_manual(values = clr_cluster) +
102  labs(subtitle = "assigned clusters") +
103  coord_fixed(xlim = c(-3,4),
104             ylim = c(-2.5,3),
105             ratio = 1) +
106  theme(axis.title.y = element_blank(),
107        axis.text.y = element_blank())

```

The last panel is going to contain the group wise summary of the cluster analysis by showing the location of the cluster centroids and the inertia based ellipses within the PCA space.

```

109 # Plot 4: Cluster centroids and inertia
110 # (sum of squared distances of samples to their closest cluster center)
111 p4 <- ggplot() +
112   geom_point(data = data_centroids,
113             aes(x = centroid_PC1, y = centroid_PC2,
114                color = factor(cluster)),
115             size = 4, shape = 21, fill = "white") +
116   geom_ellipse(data = data_centroids,
117               aes(x0 = centroid_PC1, y0 = centroid_PC2,
118                  a = semi_major_axis,
119                  b = semi_minor_axis,
120                  angle = rotation_angle, color = factor(cluster)),
121               linewidth = .2) +
122   scale_fill_manual(values = clr_alpha(clr,.2) >
123                    set_names(nm = names(clr))) +
124   scale_color_manual(values = clr_cluster) +
125   labs(x = "PC1", y = "PC2",
126        subtitle = "cluster centroids") +
127   coord_fixed(xlim = c(-3,4),
128              ylim = c(-2.5,3),
129              ratio = 1) +
130   theme(axis.title.y = element_blank(),
131        axis.text.y = element_blank())

```

The only element missing for the final figure is the illustrated legend. To be able to construct this, we first need to import the illustrations. We also create a small configuration table to specify the location of the colored circles in the legend plot.

```

133 ## --- Constructing the Legend ---
134 # Read SVG images for each species
135 images <- str_c("data/", species, ".c.png") >
136   map(read_png) >
137   set_names(nm = species)
138
139 # Define data for species positions in the legend plot
140 data_species <- tibble(species = species,
141                       y = c(.145, .5, .815),
142                       y_adj = y + c(.175,.2,.175))

```

Now we can create a last plot to construct the legend.

```

144 # Create legend plot
145 p_leg <- ggplot() +
146   geom_text(data = data_species,
147             aes(x = .67, y = y, label = species),
148             size = 4, angle = -90, family = fnt_sel) +
149   geom_circle(data = data_species,
150              aes(x0 = .5, y0 = y, r = .12,
151                 color = species,
152                 fill = after_scale(clr_alpha(color)))) +
153   annotation_custom(grob = images$schinstrap, ymin = 0.02, ymax = .31) +
154   annotation_custom(grob = images$gentoo,   ymin = .35, ymax = .64) +
155   annotation_custom(grob = images$adelie,   ymin = .68, ymax = .98) +
156   scale_color_manual(values = clrs,
157                      guide = "none") +
158   labs(caption = "artwork by @allison_horst")+
159   coord_equal(xlim = c(.3,.7),
160              ylim = c(0,1),
161                 expand = 0,
162                 clip = "off") +
163   theme_void()+
164   theme(plot.caption = element_text(size = 7,
165                                     hjust = .5,
166                                     family = fnt_sel))

```

With all the panels in place, we can finally create the whole figure.

For this we first define a kind of blue-print for the figure by creating a layout that assigns different areas to the individual panels. Then, we combine the panels using this blue-print and add the panel IDs as annotations.

```

168 ## --- Final Plot Arrangement ---
169 # Design layout for combining plots
170 design <- "AAAE
171           BCDE
172           BCDE"
173
174 # Arrange plots using patchwork
175 patchwork <- p1 + p2 + p3 + p4 +
176   p_leg +
177   plot_layout(design = design, tag_level = 'new') +
178   plot_annotation(tag_levels = list(letters[1:4], '1'))

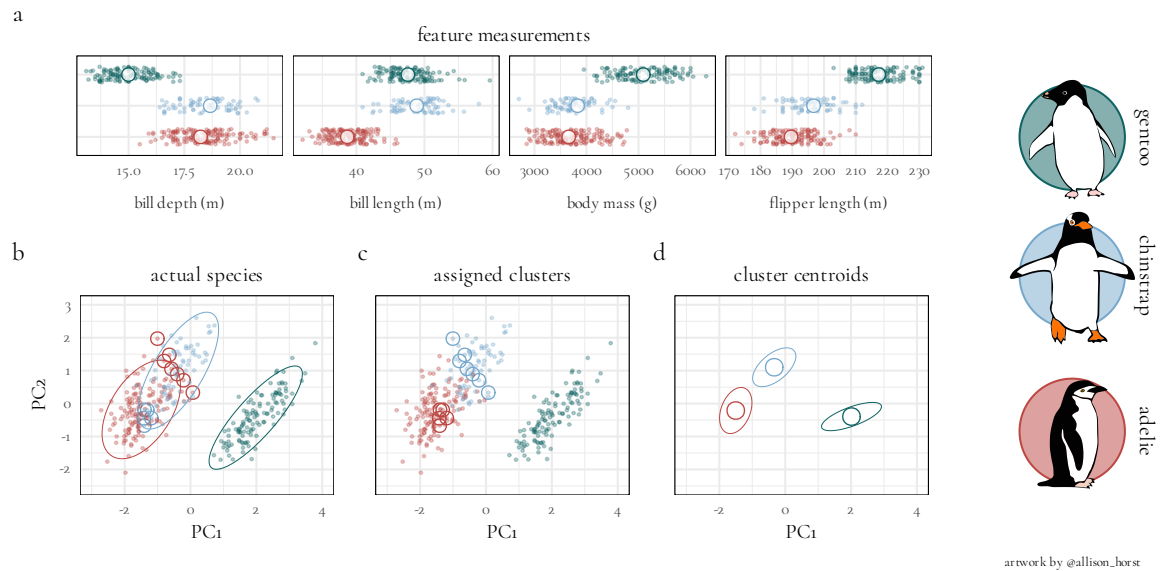
```

The figure is complete, and we can export it in pdf format.

```

180 # Save combined plot to PDF
181 ggsave(filename = "results/cluster_results.pdf",
182        plot = patchwork,
183        width = 9, height = 4.5,
184        device = cairo_pdf)

```



Finally, as our last deed of this session we record the computing environment that we used for plotting. In R, the command to do so is `sessionInfo()`, which similarly to the `python` example reports the used R version as well as the version of all used R packages.

```
186 # log session information
187 cat("R packages used:\n")
188 cat(str_c("run on the ", Sys.Date()))
189 cat("\n-----\n")
190 sessionInfo()
```

REFERENCES

Gorman, K.B., Williams, T.D. and Fraser, W.R. (2014). Ecological Sexual Dimorphism and Environmental Variability within a Community of Antarctic Penguins (Genus *Pygoscelis*). *PLOS ONE*, 9(3):e90081.